

Вопрос 20. Структурная организация памяти.
Структурная организация — это современный базовый механизм реализации виртуальной памяти.
Суть метода:
1. Все виртуальное адресное пространство каждого процесса делится на фиксированные блоки одинакового размера, называемые Виртуальными страницами (Pages). Обычно размер страницы равен 4 Кб.
2. Вся физическая оперативная память компьютера аналогично делится на блоки точно такого же размера, называемые Физическими фреймами (Frames).
3. ОС вестет для каждого процесса индивидуальную Таблицу страниц (Page Table). В этой таблице указано, какому физическому фрейму в RAM соответствует конкретная виртуальная страница.
Когда программа обращается по виртуальному адресу, процессор (модуль MMU) автоматически берет старшие биты адреса, находит по таблице страни номер физического фрейма, прибавляет к нему младшие биты (смещение внутри страницы) и получает итоговый физический адрес. Для ускорения трансляции в процессоре используется сверхбыстрый кэш — TLB (Translation Lookaside Buffer).

Вопрос 21. Механизмы виртуальной памяти и подкачки (swap).
Структурная организация позволяет загружать страницы процесса в физическую память не целиком, а по требованию. Этот механизм называется подкачкой страниц (Paging/Swapping).
Как это работает:
1. В таблице страниц для каждой записи есть специальный бит присутствия (Present bit). Если страница находится в RAM, бит равен 1.
2. Если программа пытается обратиться к виртуальной странице, которой на данный момент нет в RAM (бит равен 0), процессор приостанавливает выполнение и генерирует аппаратное исключение — Ошибку отсутствия страницы (Page Fault).
3. Ядро ОС переносит это исключение, определяет, где на жестком диске (в файле подкачки pagefile.sys или SWAP-разделе) сохранены данные этой страницы.
4. ОС находит свободный физический фрейм в RAM, считывает туда данные с диска, прописывает адрес фрейма в таблицу страниц процесса и устанавливает бит присутствия в 1.
5. Процессор повторно выполняет инструкцию, вызвавшую ошибку, и программа продолжает работу прозрачно для себя. Если свободной RAM нет, ОС предпочтительно выгружает неиспользуемую страницу на диск.

Вопрос 22. Алгоритмы записи страниц.
Когда возникает Page Fault, а в физической оперативной памяти нет ни одного свободного фрейма, операционная система должна выбрать, какую из текущих страниц необходимо выгрузить (заменить) на жесткий диск с SWAP.
Для выбора страниц используются специальные алгоритмы:
1. FIFO (First-In, First-Out): Вытесняются страницы, которая была загружена в память раньше всех. Алгоритм прост, но неэффективен, так как может выгрузить часто используемую базовую страницу. Подвержен аномалии Бездна (при увеличении количества физической памяти число ошибок страниц может вырасти).
2. LRU (Least Recently Used — Наиболее давно использовавшаяся): Выгружаются страницы, к которой не было обращений дольше всего. Математически это оптичный алгоритм, основанный на принципе локальности данных, но его чистая реализация требует огромных аппаратных затрат (счетовых времени для каждой строки).
3. Алгоритм Clock (Часы / Вторая попытка): Эффективная и дешевая аппроксимация LRU. Все фреймы организованы в логический круг, по которому движется стрелка. У каждой страницы есть бит обращения (Reference bit), устанавливаемый процессором в 1 при любом доступе. Если стрелка наткнется на страницу с битом 1, она сбрасывает его в 0 и идет дальше (давая странице вторую шанс). Если стрелка встретилась с страницей с битом 0, эта страница немедленно выбирается на выгрузку.

Вопрос 23. Понятие фрагментации памяти и методы ее уменьшения.
Фрагментация памяти — это неэффективное использование оперативной памяти, при котором чистый объем свободной памяти может быть большим, но выделить ее под новые задачи становится невозможно. Выделяют два вида фрагментации:
1. Внешняя фрагментация: Возникает при динамическом выделении участков произвольной длины (например, при сегментной организации). Свободная память, дробится на множество мелких изолированных блоков, чередующихся с занятыми участками. Если процессу нужен один сплошной кусок в 10 Мб, а у нас есть с 5 фрагментов по 3 Мб, система ответит отказом, хотя суммарно свободно 15 Мб. Методы уменьшения: Сжатие (дефрагментация) памяти путем перемещения занятых блоков в один конец (ресурсоемо) или полный переход на страничную организацию, где физические фреймы могут выделяться произойно.
2. Внутренняя фрагментация: Возникает, когда память выделяется фиксированными блоками (страницами). Если процессу требуется всего 1 Кб памяти, ОС все равно выделит ему целую страницу размером 4 Кб. Оставшиеся 3 Кб внутри этой страницы оказываются потерянными и не могут быть отданы другим процессам. Методы уменьшения: Использование внутрипроцессных динамических аллокаторов памяти (например, jemalloc, tcmalloc), которые дробят крупные страницы ядра на мелкие пулы объектов фиксированного размера.

Вопрос 24. Управление ресурсами и контроль использования памяти процессами.
Операционная система должна жестко лимитировать объемы ресурсов, чтобы один монопольный или зависящий процесс не истерпал всю память устройства, парализовав работу.
Для контроля ОС использует механизмы квот (Working Set in Windows или rlimit in Linux). ОС отслеживает размер резидентной памяти процесса (RSS) — объем страни, реально находящихся в данный момент в ОЗУ.
В критических ситуациях, когда физическая RAM и все пространство SWAP полностью заполнены, в современных ОС срабатывают защитные модули аварийного завершения. Например, в ядре Linux функционирует алгоритм OOM Killer (Out-Of-Memory Killer). Он рассчитывает для каждого процесса балл штрафа (badness score) на основе того, сколько памяти занимает процесс, как долго он живет и запущен ли он от root. Процесс с наивысшим баллом принудительно уничтожается по сигналу SIGKILL. Для спасения системы вынуждены всей ОС.
Вопрос 25. Назначение файловой системы.
Файловая система (ФС) — это системная структура и совокупность алгоритмов, определяющих способ организации, хранения, именования и разграничения доступа к файлам на физических носителях информации (HDD, SSD, flash-накопители). Назначение:
— Предоставление пользователям и приложениям удобного абстрактного интерфейса доступа к данным в виде древовидной структуры файлов и каталогов (вместо работы с сырыми номерами секторов и дорожек диска);
— Оптимизация распределения дискового пространства и минимизация фрагментации файлов;
— Обеспечение надежности и сохранности данных при сбоях электропитания (с помощью журналирования);
— Сокрытие специфики физического накопителя от прикладного ПО.

Вопрос 26. Основные компоненты файловой системы.
Любая файловая система состоит из следующих ключевых структурных компонентов:
1. Суперблок (или Главная файловая таблица / Загрузочный сектор): Находится в начале раздела и содержит глобальные метаданные о самой ФС: общий размер, размер одного блока, количество свободных и занятых блоков, тип ФС и флаг состояния.
2. Структуры управления файлами (Имена в Linux/ext4, запись MFT в Windows/NTFS): Это служебные записи, закрепленные за каждым файлом. Они не содержат имя или само тело файла, но хранят его метаданные: размер, дату создания/изменения, права доступа, ID владельца и адреса физических блоков диска, на которых записано содержимое.
3. Блоки данных: Непосредственная область диска, разбитая на кластеры, где хранятся байты содержимого файлов.
4. Каталог (Directory): Специальные файлы, которые содержат таблицы соответствия между понятными человеку именами файлов и их внутренними номерами управляющих блоков (инодов).

Вопрос 27. Типы файловых систем и их особенности (FAT, NTFS, ext4).
1. FAT32 (File Allocation Table): Историческая система.
— Особенности: Использование простой таблицы распределения файлов. Отсутствуют встроенные механизмы разграничения прав доступа и журналирование.
— Ограничения: Максимальный размер одного файла не может превышать 4 Гб. Максимальный размер раздела — до 2 Тб. Обладает высокой кроссплатформенной совместимостью, потому что сик пр применяется на флешках.
2. NTFS (New Technology File System): Стандарт для ОС Windows.
— Особенности: Журналируемая ФС (при записи сигнала фиксируется лог операции, что предотвращает повреждение ФС при внезапном отключении питания). Поддерживает разграничение прав доступа через списки ACL, прозрачное сжатие данных, шифрование (EFS), кэпирование и жесткие ссылки.
3. ext4 (Fourth Extended Filesystem): Стандарт для большинства дистрибутивов Linux.
— Особенности: Высокопроизводительная журналируемая ФС. Использует механизм экстендов (блоков непрерывных секторов), что резко снижает внешнюю фрагментацию файлов. Поддерживает файлы размером до 16 Тб и разделы до 1 Эб.

Вопрос 28. Организация доступа к файлам и каталогам.
Организация доступа к данным базируется на иерархическом обходе дерева каталогов. Когда приложение выполняет системный вызов на открытие файла по имени пути (например, /home/user/data.txt), операционная система производит пошаговое разрешение имени:
1. Ядро обращается к корневому каталогу "/" и считывает его блок данных.
2. Внутри ядра ОС ищет строку 'home' и узнает номер её инода.
3. ОС читает инод папки 'home', открывает её блок данных, ищет там запись 'user' и получает номер её дескриптора.
4. Процедура повторяется до тех пор, пока ядро не найдет инод целевого файла 'data.txt'.
Внутри инода проверяются права доступа текущего пользователя, и если доступ разрешен, создается файловый дескриптор, возвращаемый программе для чтения/записи.

Вопрос 29. Права доступа и модели безопасности файловых систем.
Для защиты данных от несанкционированного доступа ОС поддерживает модели безопасности:
1. Модель user (POSIX / Linux): Каждому файлу сопоставляются права для трех категорий пользователей: Владелец (user), Группа владельца (group) и Все остальные (Others). Для каждой категории задются три права: Read (чтение — 4), Write (запись — 2), Execute (выполнение — 1). Права записываются в виде триад или восьмеричных чисел (например, 755).
2. Списки контроля доступа (ACL — Access Control Lists / Windows и современные Unix): Более гибкая и детальная модель. К файлу привязывается таблица, где для каждого конкретного пользователя или группы явно перечисляются разрешения или запрещенные действия (например, "Пользователь Иванову разрешено чтение, но запрещено удаление", Группе Бухгалтеры разрешено изменение).
Основные типы доступа: Дискреционное управление (DAC — владелец сам решает права) и Мандатное управление (MAC — права спускаются централизованно системой на основе уровней секретности).

Вопрос 30. Кшифрование и буферизация ввода-вывода.
Дисковые накопители работают в тысячи раз медленнее оперативной памяти. Для сокрытия этой разницы ОС применяет механизмы буферизации и кшифрования:
— Буферизация (Buffering): Использование выделенной области памяти (буфера) для временного накопления порций данных. Например, если программа пишет на диск по одному байту, ОС собирает эти байты в буфер размером 4 Кб и сбрасывает их на накопитель одним сплошным блоком, что снижает число дорогостоящих обращений к аппаратуре.
— Кшифрование страниц (Page Cache): Механизм, при котором операционная система оставляет в свободной оперативной памяти копии блоков файлов, считанных с диска ранее. Если эти же или другие приложение снова запрашивает доступ к тем же данным, ОС мгновенно отдает их из ОЗУ, полностью избегая обращения к физическому диску. Записи также могут кшифроваться (отложенная запись), когда данные фиксируются в RAM, а на диск переносятся фоновыми потоками ядра позже.

Вопрос 31. Понятие RAID и его основные уровни.
RAID (Redundant Array of Independent Disks — Избыточный массив независимых дисков) — это технология объединения нескольких физических жестких дисков или SSD в один логический элемент для повышения надежности хранения данных, увеличения скорости работы или объединения емкостей. Основные уровни RAID:
— RAID 0 (Чередование / Striping): Данные разбиваются на блоки и записываются поочередно на все диски массива параллельно. Плюсы: Максимальная скорость чтения и записи. Минусы: Надежность падает пропорционально числу дисков. Отказ любого одного диска безвозвратно уничтожает абсолютную все данные массива.
— RAID 1 (Зеркалирование / Mirroring): Данные полностью дублируются (копируются) на два или более дисков одновременно. Плюсы: Высокая надежность. Массив функционирует, пока жив хотя бы один диск. Минусы: Высокая стоимость (потеря половины емкости).
— RAID 5 (Чередование с распределенной контрольной суммой): Требует минимум 3 диска. Данные и блоки четности (паритета) циклически распределяются по всем накопителям. Плюсы: Высокая скорость чтения, экономическая эффективность. Выдерживает полный отказ любого одного диска без потери данных (информация восстанавливается на лету за счет математического расчета паритета).

Вопрос 32. Архитектура операционной системы Windows (HAL, ядро NT).
Архитектура современных ОС семейства Windows (начиная с NT и заканчивая Windows 11) является сложной гибридной системой. В ней выделяют два пространства: режим пользователя и режим ядра. Ключевые компоненты режима ядра:
1. HAL (Hardware Abstraction Layer — Слой абстракции от оборудования): Это самый низкоуровневый программный слой, который напрямую взаимодействует с материнской платой и процессором. Он полностью скрывает специфику конкретного железа (архитектуру чипсетов, контроллеров прерываний) от вышележащего ядра, предоставляя ему единый API. Благодаря HAL код Windows переносим.
2. Микроядро Windows NT: Занимается исключительно базовыми задачами: диспетчеризацией потоков, обработкой прерываний и синхронизацией процессоров.
3. Исполнительная система (Executive): Набор крупных модулей, расположенных над ядром. Сюда входит Менеджер объектов (Object Manager), Менеджер памяти (Memory Manager), Менеджер процессов (Process Manager), Подсистема безопасности (SRM). В Windows эти компоненты работают в режиме ядра ради высокой производительности, что делает архитектуру гибридной.

Вопрос 33. Роль реестра Windows и его структура.
Регистр Windows (Windows Registry) — это централизованная, структурированная иерархическая база данных, используемая операционной системой для хранения всех конфигурационных настроек, параметров и профилей.
Роль реестра: В нем содержатся настройки для аппаратного обеспечения, драйверов устройств, самой операционной системы, параметров безопасности и настроек всех установленных пользовательских приложений. Он заменяет собой множество разрозненных конфигурационных файлов .ini.
Структура реестра имитирует файловую систему и состоит из разделов (Keys), подразделов и параметров (Values).
Корневые разделы реестра:
— HKEY_LOCAL_MACHINE (HKLM): Содержит глобальные настройки компьютера, одинаковые для всех пользователей (список оборудования, драйверы, системные политики, общие программы);
— HKEY_CURRENT_USER (HKCU): Хранит настройки профиля текущего пользователя, совершившего вход (тема оформления, обои, личные параметры софта);
— HKEY_CLASSES_ROOT (HKCR): Отвечает за ассоциации типов файлов (каким приложением открывать .txt, .pdf) и регистрацию COM-компонентов;
— HKEY_USERS (HKU): Содержит профили всех пользователей системы.

Вопрос 34. Процессы и службы в Windows.
В Windows все исполняемые фоновые и интерактивные элементы четко разделены:
1. Пользовательские процессы: Запускаются пользователем лично или от его имени при входе в систему. Не взаимодействуют внутри интравиртуальной сессии, имеют графический (GUI) или консольный интерфейс (например, chrome.exe, calc.exe, explorer.exe). Завершаются при выходе пользователя из системы.
2. Службы (Windows Services): Специализированные фоновые процессы, спроектированные для выполнения системных задач без взаимодействия с пользователем. Они запускаются автоматически еще до ввода пароля на экране блокировки и работают в фоновом режиме в изолированной неинтерактивной сессии (Session 0). Службы не имеют права выводить окна на экран.
Примеры служб: служба планировщика задач, служба обновлений Windows (wuauclnt), брандмауэр, базы данных. Управлением службами занимается системный диспетчер SCM (Service Control Manager).

Вопрос 35. Средства мониторинга процессов (Task Manager, Process Explorer).
Для контроля за состоянием операционной системы, запущенных программ и утилитации ресурсов применяются инструменты мониторинга:
1. Диспетчер задач (Task Manager): Стандартный инструмент Windows. Позволяет быстро оценить суммарную загрузку CPU, RAM, дисковой подсистемы, видеокарты и сети. Предоставляет список запущенных приложений и служб, позволяет анализировать автозагрузку программ и принудительно завершить («снять задачу») зависшие процессы.
2. Процесс Эксплорер: Профессиональный инструмент утилиты от Майкрософт (входит в пакет Sysinternals). Обладает колоссальными возможностями по сравнению с Task Manager:
— Отображает процессы в виде наглядного иерархического дерева (видно, какой процесс какой породил);
— Позволяет просмотреть список всех открытых процессов файлов, папок, ключей реестра (Handles);
— Показывает все загруженные процессом динамические библиотеки (DLL);
— Позволяет детально изучать тект каждого потока внутри выбранного процесса, что незаменимо при отладке и поиске вирусов.

Вопрос 36. Командная строка и терминальные средства Windows.
Интерфейс командной строки (CLI) необходим для автоматизации и администрирования.
1. Командная строка (cmd.exe): Традиционный интерпретатор команд Windows. Он является прямым наследием операционной системы MS-DOS и командного процессора command.com. Обладает ограниченным синтаксисом, ориентирован на работу с текстовым выводом и запуск простейших пакетных скриптов (.bat/.cmd). Медленно устаревает.
2. Windows Terminal: Современное многофункциональное терминальное приложение от Microsoft. Оно не заменяет CMD или PowerShell, а служит единой графической оболочкой («контейнером») для них. Windows Terminal поддерживает вкладки, профили конфигурации, красивые темы оформления, аппаратное ускорение рендеринга текста через GPU и позволяет в рамках одного окна одновременно держать открытыми вкладки классической CMD, PowerShell, Azure Cloud Shell и дистрибутивов Linux (WSL).

Вопрос 38. PowerShell как средство автоматизации администрирования.
PowerShell — это расширенное объектно-ориентированное средство автоматизации от Microsoft, состоящее из оболочки командной строки и специализированного языка сценариев, построенного на базе .NET Framework.
Главное фундаментальное отличие PowerShell от Bash или CMD: Традиционные оболочки работают с текстом. Результаты выполнения команды в Linux — это поток строк, который нужно парсить утилитами grep, awk, sed.
PowerShell полностью объектно-Ориентирован. Команды PowerShell возвращают не текст, а полноценные объекты .NET, обладающие строго типизированными свойствами и методами. Команды в PowerShell называются команда-методами (cmdlets) и имеют интуитивную структуру имен: Типаго-Сущестительное (например, Get-Process, Stop-Service, New-Item). Передавая объекты по конвейеру (pipe '|'), администратор может легко отфильтровать процессы по объекту памяти (например, 'Get-Process | Where-Object {\$_.WorkingSet -gt 100MB}'), обращая напрямую к свойствам объекта, без парсинга текста.
Вопрос 38. Сценарии PowerShell и области их применения.
Сценарии (скрипты PowerShell) сохраняются в файлы с расширением '.ps1'. Они позволяют объединять цепочки команд в сложные автоматизированные программы с поддержкой циклов, условий, обработки исключений и функций.
Области применения сценариев PowerShell:
— Массовое администрирование пользователей и прав в корпоративных доменах Active Directory;
— Настройка, развертывание и управление облачными ресурсами (Azure, AWS) и виртуализацией Hyper-V;
— Сбор системных логов, мониторинг состояния серверов и автоматическое резервное копирование данных;
— Удаленное управление тысячами компьютеров через встроенный безопасный протокол PS Remoting / WinRM (команда выполняется локально на сервере, а результат-объект возвращается администратору).

Вопрос 39. Архитектура ОС Linux.
Linux является UNIX-подобной операционной системой, спроектированной на принципах стандартов POSIX. Архитектура системы строго разделена на две изолированные области памяти:
1. Пространство пользователя (User Space): Здесь выполняются все пользовательские приложения, графические оболочки (GNOME, KDE) и системные утилиты. Программы общаются с аппаратной частью не напрямую, а через стандартную системную библиотеку языка C ('libc'), которая берет на себя формирование системных вызовов.
2. Пространство ядра (Kernel Space): Область памяти, где выполняется ядро Linux. Оно полностью изолировано и защищено от пользовательских программ. В ядре сосредоточены подсистемы управления процессами (планировщик), памяти, виртуальной файловой системы (VFS), сетевого стека и драйверов. Такое четкое разделение гарантирует, что падение любого пользовательского приложения (например, зависание видеоплеера) никак не нарушит стабильности работы ядра и других программ.

Вопрос 40. Особенности мониторинга ядра Linux.
Ядро Linux является классическим Монолитным ядром. Это означает, что все его ключевые подсистемы и драйверы работают в едином адресном пространстве режима ядра, обеспечивая максимальное быстродействие.
Однако у ядра Linux есть важнейшая особенность, которая нивелирует главный недостаток классических монолитов — Динамические загружаемые модули ядра (LKM — Loadable Kernel Modules). В Linux ядро не является жестко скомпилированным статическим блоком. Оно модульное. Драйверы для новых устройств, файловые системы или сетевые протоколы могут быть скомпилированы в виде отдельных файлов-модулей ('.ko') и загружены в память ядра прямо на лету во время работы системы (командами 'modprobe' или 'insmod') без необходимости перезагрузки компьютера. Если устройство отключается, модуль автоматически выгружается ('rmmod'), освобождая оперативную память. Это дает Linux гибкость микроядра при сохранении скорости монолита.

Вопрос 41. Виртуальная файловая система (VFS) в Linux.
Виртуальная файловая система (VFS) — это абстрактный слой ядра Linux, расположенный между прикладными программами и конкретными реализациями файловых систем (ext4, NTFS, FAT, NFS). Назначение VFS: Предоставление приложениям единого, унифицированного интерфейса для работы с файлами. Для программиста операции открытия файла, чтения или записи выглядят абсолютно одинаково, независимо от того, где физические расположены данные.
VFS берет на себя перевод стандартных системных вызовов (open, read, write) в специфические функции и алгоритмы конкретной файловой системы, на которой смонтирован каталог. Более того, благодаря VFS в Linux проточено функционирование псевдофайловых систем (такие как '/proc' и '/sys'), которые физически не существуют на диске, а генерируются ядром прямо в оперативной памяти для отображения системной информации.

Вопрос 42. Файловая система Linux и иерархия каталогов.
В Linux отсутствует понятие логических дисков (C:, D:), привычных для Windows. Все устройства монтируются в единое дерево каталогов, корнем которого является каталог '/'.
Иерархия папок строго регламентирована международным стандартом FHS (Filesystem Hierarchy Standard).
— '/bin' и '/sbin': Содержат критически важные исполняемые файлы и утилиты, необходимые для работы системы и администратора (ls, cp, rm, fdisk);
— '/etc': Главное хранилище текстовых конфигурационных файлов операционной системы и всех установленных программ;
— '/dev': Папка файлов устройств. В Linux реализуется принцип 'все есть файл'. Каждому аппаратному устройству соответствует файл (например, '/dev/sda' — первый жесткий диск);
— '/var': Динамическое хранилище обычных пользовательских системных (личные документы, настройки софта);
— '/usr': Содержит переносимые данные, размер которых постоянно меняется в процессе работы: логи системы (журналы), бинарные данные, кэш;
— '/proc' и '/sys': Виртуальные папки (интерфейсы к ядру). Например, прочитав файл '/proc/cpuinfo', можно узнать модель процессора.

Вопрос 43. Права доступа в Linux (chmod, chown, umask).
Модель безопасности Linux базируется на дискреционном управлении доступом. Права проверяются для трех категорий:
— Y (User) — владелец файла;
— G (Group) — пользователи, входящие в группу файла;
— O (Others) — все остальные пользователи в системе.
Для каждого каталога задются три типа прав: Read (r — чтение, численно 4), Write (w — запись, численно 2), Execute (x — выполнение/запуск для файлов или просмотр содержимого для каталогов, численно 1).
Инструменты управления:
— 'chmod': Изменяет права доступа. Можно задавать символами ('chmod u+x script.sh' — добавить право запуска владельцу) или числовыми масками ('chmod 755 file' — владельцу все права +4+2+1=7, группе и остальным только чтение и запуск +4+1=5).
— 'chown': Позволяет изменить владельца и/или группу файла (например, 'chown root:admins /etc/secret.conf').
— 'umask': Маска режима создания файлов. Это специальное числовое значение, определяющее, какие права будут автоматически заблокированы (вычтены из максимальных 666 для файлов и 777 для папок) у любого нового создаваемого объекта.

Вопрос 44. Командная оболочка Bash и ее возможности.
Bash (Bourne-Again Shell) — это стандартный командный интерпретатор (оболочка), используемый по умолчанию в большинстве дистрибутивов Linux.
Ключевые возможности Bash:
— Конвейеризация (Pipes '|'): Позволяет перенаправлять стандартный вывод одной команды напрямую на вход другой команды, выстраивая мощные цепочки обработки информации (например, 'cat log.txt | grep ERROR | wc -l' — отфильтровать ошибки и посчитать их количество);
— Перенаправление ввода-вывода ('>', '>>', '<'): Позволяет перенаправить результат команды в файл или читать данные из файла вместо клавиатуры;
— Автодополнение: Нажатие клавиш Tab автоматически дописывает имена команд, пути к файлам и параметры;
— Язык программирования сценариев: Поддерживает переменные, циклы (for, while), ветвления (if-then-else) и функции. Позволяет создавать bash-скрипты для полной автоматизации обслуживания сервера.

Вопрос 45. Управление процессами в Linux (fork, exec, nice, kill).
В Linux создание и управление процессами реализовано через эвентную модель системных вызовов:
— 'fork()': Системный вызов создания нового процесса. Он не запускает программу с нуля, а делает точную копию текущего родительского процесса в памяти. Новый процесс (child) получает свой PID, но наследует все переменные и открытые файлы родителя;
— 'exec()': Семейство вызовов, которое полностью заменяет текущий код и адресное пространство процесса новой программой, загруженной с диска. Обычно в Linux новый процесс запускается дутом: сначала вызывается 'fork()', а затем внутри дочернего процесса вызывается 'exec()';
— 'nice': Утилита и системный вызов для управления приоритетом планирования процесса. Значение nice варьируется от -20 (наивысший приоритет, процесс 'монстринг' и забирает CPU) до 19 (наинизший приоритет, процесс максимально 'миа' и уступает CPU другим);
— 'kill': Системный вызов и утилита для отправки сигналов процессам. Несмотря название, она не только убивает. Сигнал 'SIGTERM (15)' просит процесс корректно завершиться самостоятельно, а сигнал 'SIGKILL (9)' немедленно и принудительно уничтожает процесс на уровне ядра.

Вопрос 46. Межпроцессное взаимодействие (IPC): каналы, сигналы, сокеты.
Так как процессы в Linux полностью изолированы друг от друга на уровне адресных пространств виртуальной памяти, для их общения применяются специальные механизмы IPC (Inter-Process Communication):
1. Анонимные каналы (Pipes): Однонаправленный буфер в памяти ядра. Данные, записанные одним процессом, считываются другим в порядке очереди (FIFO). Используются в командной строке через символ '|'.
2. Сигналы (Signals): Метод асинхронного уведомления. Ядро передает процессу короткий числовой код (сигнал) при наступлении события (например, нажатие Ctrl+C посылает сигнал SIGINT). Процесс прерывает работу и вызывает свой обработчик сигнала.
3. Сокеты (Sockets): Наиболее гибкий механизм.
— Сетевые сокеты (IP/TCP/UDP) позволяют процессам обмениваться двунаправленными потоками байт как в рамках одного компьютера, так и через Интернет;
— Локальные сокеты (Unix Domain Sockets) используют интерфейс файлов на диске, но обмен идет целиком внутри оперативной памяти ядра, что обеспечивает колоссальную скорость локального взаимодействия.

Вопрос 47. Многопоточность в Linux (pthreads).
Архитектурно ядро Linux уникально тем, что внутри него нет жесткого разделения на процессы и потоки. Ядро оперирует абстракцией 'задача' (task, struct).
Потоки в Linux реализованы как Легковесные процессы (LWP — Light-Weight Processes). Они создаются с помощью универсального системного вызова 'clone()'. Флагами вызова 'clone()' ядру указывается, какие именно ресурсы новая задача должна разделять с родительской. Если передать флаги совместного использования памяти, файловой системы и сигналов — рождается поток.
Для написания переносимых многопоточных программы на языках C/C++ используется международный стандарт POSIX Threads, реализуемый системной библиоткой 'libpthread'. Она предоставляет разработчику готовый набор функций для создания потоков ('pthread_create'), ожидания их завершения ('pthread_join') и примитивов синхронизации (мьютексы 'pthread_mutex_t', условные переменные).

Вопрос 48. Systemd и управление службами.
Systemd — это современная системная подсистема инициализации и управления демонами (службами) в Linux, которая запускается ядром первой (получает PID 1) и координирует всю работу ОС. Она заменила старую систему SysV init.
Главная особенность Systemd — Агрессивный параллельный запуск служб. Если старая система запускала службы строго поочередно (что замедляло загрузку), Systemd создает сокеты служб заранее и запускает их параллельно, ускоряя старт системы.
Объекты управления в systemd называются Юнитами (Units), их конфигурация описывается простыми текстовыми файлами (например, 'nginx.service'). Основные типы юнитов: 'service' (службы), 'target' (группы юнитов/уровни загрузки), 'timer' (аналог cron).
Главный инструмент администратора — утилита 'systemctl':
— 'systemctl start имя' / 'stop' — запустить или остановить службу;
— 'systemctl enable имя' / 'disable' — добавить или убрать службу из автозагрузки при старте ПК;
— 'systemctl status имя' — детально проверить состояние службы и прочитать последние строчки её логов из встроеного бинарного журнала 'journald'.

Вопросы ОС зачёт
1. Назначение операционной системы и её место в архитектуре вычислительной системы.
2. История и эволюция операционных систем: основные этапы развития.
3. Классификация операционных систем (по назначению, режиму работы, архитектуре).
4. Архитектура вычислительной системы: взаимодействие аппаратного обеспечения и ОС.
5. Понятие ядра операционной системы. Режимы User mode и Kernel mode.
6. Типы архитектур ядер: монолитное, микроядерное, гибридное — сравнительный анализ.
7. Системные вызовы: назначение, механизмы работы, примеры.
8. Прерывания и исключения: различия, роль в работе ОС.
9. Понятие процесса и потока. Их различия и области применения.
10. Состояния процесса и жизненный цикл процесса.
11. Планирование процессорного времени: цели и критерии эффективности.
12. Алгоритмы планирования (FIFO, Round Robin, приоритетное планирование).
13. Многозадачность и многопоточность: преимущества и ограничения.
14. Синхронизация процессов и потоков.
15. Примитивы синхронизации: мьютексы, семафоры, спинлоки.
16. Проблема взаимной блокировки (deadlock): условия возникновения и методы предотвращения.
17. Классические задачи синхронизации (проблема «обедующие философы»);
18. Назначение подсистемы управления памятью в ОС.
19. Физическая и виртуальная память: основные отличия.
20. Страничная организация памяти.
21. Механизмы виртуальной памяти и подкачки (swap).
22. Алгоритмы замещения страниц.
23. Понятие фрагментации памяти и методы её уменьшения.
24. Управление ресурсами и контроль использования памяти процессами.
25. Назначение файловой системы.
26. Основные компоненты файловой системы.
27. Типы файловых систем и их особенности (FAT, NTFS, ext4).
28. Организация доступа к файлам и каталогам.
29. Права доступа и модели безопасности файловой систем.
30. Копирование и буферизация ввода-вывода.
31. Понятие RAID и его основные уровни.
32. Архитектура операционной системы Windows (HAL, ядро NT).
33. Роль реестра Windows и его структура.
34. Процессы и службы в Windows.
35. Средства мониторинга процессов (Task Manager, Process Explorer).
36. Командная строка и терминальные средства Windows.
37. PowerShell как средство автоматизации администрирования.
38. Сценарии PowerShell и области их применения.
39. Архитектура ОС Linux.
40. Особенности монолитного ядра Linux.
41. Виртуальная файловая система (VFS) в Linux.
42. Файловая система Linux и иерархия каталогов.
43. Права доступа в Linux (chmod, chown, umask).
44. Командная оболочка Bash и её возможности.
45. Управление процессами в Linux (fork, exec, nice, kill).
46. Межпроцессное взаимодействие (IPC): каналы, сигналы, сокеты.
47. Многопоточность в Linux (pthreads).
48. Systemd и управление службами.